STREFA K-12 · DOKUMENTACJA PROJEKTOWA · 09

Dla Claude AI

Instrukcja dla kolejnych sesji Claude rozwijających Strefę K-12: środowisko, architektura kodu, konwencje, testy, wydania, pułapki techniczne i sposób pracy z autorem.

ДЛЯ СЛУЖЕБНОГО ПОЛЬЗОВАНИЯ

Stan gry: v0.16.6 (październik 2026)
Godot 4.6.1 · renderer Compatibility · GDScript
Repozytorium: github.com/budownik1203/k-12

Spis treści

- Zacznij tutaj
- Autor i sposób pracy
- Architektura
  - Autoloady (scripts/autoload/)
  - Sceny
  - Katalogi
  - Kluczowe kontrakty w kodzie
- Konwencje kodu
- Testy
  - Uruchamianie
  - Tryby (scenes/rajd.tscn -- …)
  - Regresja
  - Weryfikacja wizualna
- Wydanie
- Znane pułapki techniczne
- Jak dodać…
- Otwarte sprawy (stan na v0.16.6)

Zacznij tutaj

1. Przeczytaj `docs/PRZEKAZANIE.md` (krótka wersja tego dokumentu), potem `docs/STAN_DEMO.md` (tabela wszystkiego, co zrobione) i ten dokument.
2. Przygotuj środowisko: `bash narzedzia/przygotuj_srodowisko.sh`, potem `export PATH=$HOME/tools:$PATH`.
3. Sprawdź parsowanie: `godot --headless --path . tests/sprawdz.tscn` → „SPRAWDZONO SKRYPTOW: 63” bez błędów.
4. Odpowiadaj autorowi **po polsku**. Kod, komentarze i dokumentacja też po polsku.

Autor i sposób pracy

- Autor (użytkownik) pisze krótko, często z literówkami, po polsku. Zgłasza błędy **zrzutami ekranu z gry** (Windows, 1920×1080).
- Oczekuje **pełnej samodzielności**: wdrożenie → testy → wydanie zipa → krótki raport po polsku z linkiem. Nie pyta o szczegóły implementacji.
- Prosi o **propozycje** (mechaniki, poprawki); zwykle odpowiada „wszystko oprócz …” albo „+ to co proponujesz”. Dobra praktyka: na końcu raportu 3–5 konkretnych pomysłów zgodnych z filarami.
- **Filary, których pilnuje:** gra wolna, ciężka, realistyczna; bez wygodnych skrótów; bez informacji, której postać nie ma; rzeczy fizyczne obsługuje ręka. Odrzucił: leżenie, wstrzymanie oddechu, znaki gracza między rajdami.
- Gdy autor zgłasza „błąd renderowania”, najpierw **zreprodukuj** (test, zrzut z xvfb), dopiero potem poprawiaj; jeśli przyczyna niepewna – powiedz to wprost i poproś o zrzut z innej perspektywy.

Architektura

Autoloady (`scripts/autoload/`)

Nazwa	Rola
`Gra` (gra.gd)	stan gry: bony, ekwipunek, skrytka, kryjówka, reputacja, zlecenia, zdrowie, potrzeby, ustawienia; zapis/odczyt JSON; przejścia scen; meldunki; `emituj_halas`; `wskazowka` (samouczek); pełny ekran (F11)
`Dane` (dane.gd)	wczytuje `przedmioty.json` i `meble.json`; definicje, ikony, tabele łupu
`Dzwiek` (dzwiek.gd)	odtwarzanie 2D/3D, warianty, pogłos, okluzja

Nazwa	Rola
`Sterowanie` (sterowanie.gd)	akcje klawiszy, `blokada` (UI otwarte), przechwytywanie myszy
`PsxMaterialy`	materiały i shadery

Sceny

Scena	Skrypt	Opis
`scenes/menu.tscn`	`scripts/ui/scena_menu.gd`	menu główne + panel ustawień
`scenes/baza.tscn`	`scripts/ui/scena_baza.gd`	placówka; `scripts/swiat/kryjowka.gd` – kryjówka i tryb budowy
`scenes/rajd.tscn`	`scripts/ui/scena_rajd.gd`	rajd: generowanie kondygnacji, wrogowie, wyjścia, światła, **większość testów**
`scenes/raport.tscn`	`scripts/ui/scena_raport.gd`	raport po rajdzie

Katalogi

Katalog	Zawartość
`scripts/core/`	dane i logika bez sceny: `przedmiot`, `siatka`, `ekwipunek`, `cialo`, `balistyka`, `frakcje_bazy`, `meble_kryjowki`, modele (`modele*.gd`, `bryly.gd`, `kit.gd` – budowa siatek), `widok.gd` (SubViewport świata)
`scripts/gracz/`	`gracz.gd` (ruch, interakcje, chwyt, ciało, stres), `bron.gd` (broń, animacje, latarka), `przeladowania.gd` + `choreografia.gd` (przeładowania jako sekwencje), `animacje_akcji.gd` (czynności), `reka.gd` (dłoń z palcami, IK łokcia)
`scripts/ai/`	`wrog.gd` (AI ludzi), `menedzer_wrogow.gd` (oddziały, fale, boss, polowanie), `fauna.gd` (pełzacz), `ragdoll.gd`
`scripts/swiat/`	`generator.gd`, `drzwi.gd`, `pojemnik.gd`, `przesuwny.gd`, `pulapka.gd`, `potykacz.gd`, `wylacznik.gd`, `ewakuacja.gd`, `granat.gd`, `efekty.gd`, `naklejki.gd`, `lup_na_ziemi.gd`, `modele_mebli.gd`, `migotanie.gd`
`scripts/ui/`	`hud.gd`, `ekran_ekwipunku.gd` (największy, ~3500 linii), `siatka_ui.gd`, `handel_ui.gd`, `menu_pauzy.gd` (wspólna lista ustawień), `wytrych_ui.gd`, `modele_oporzadzenia.gd`, `rozlozenie_3d.gd`, `panel_ciala.gd`
`assets/dane/`	`przedmioty.json` (przedmioty, tabele łupu, frakcje), `meble.json`
`narzedzia/`	generatory zasobów w Pythonie (tekstury, dźwięki, ikony, modele, naklejki), skrypty środowiska, regresji, wydania i dokumentacji
`tests/`	`sprawdz` (parsowanie), `test_ekwipunek`, `test_kontrakty`, `test_ui`, `petla` (pełna pętla gry), `test_sciany` (generator)

Kluczowe kontrakty w kodzie

- **Interakcja:** kolizja z meta `"interakcja"` wskazuje obiekt z `tekst_interakcji(g) -> String` i `interakcja(g)`. Chwytne obiekty mają `chwyc(g)`, `chwyc_ruch(g, m: Vector2, delta)`,

`pusc_chwyt(g)`, opcjonalnie `mozna_chwycic(g)` i właściwość `przejmuje_mysz` (mysz steruje obiektem zamiast kamerą). HUD dodaje „[F]” tylko, gdy tekst go nie zawiera.

- **Czynności gracza:** `gracz.zacznij_akcje(nazwa, czas, koniec: Callable, typ_animacji, kroki, na_krok, dane)` – czynność trwa, ma animację, przerywa ją X, strzał, trafienie.
- **Hałas:** `Gra.emituj_halas(pozycja, zasieg, kto)`; AI liczy drogę dźwięku po nawigacji (`Balistyka.odleglosc_dzwieku`).
- **Wskazówki samouczka:** `Gra.wskazowka(id, tekst)` – każda raz.
- **Warstwy fizyki:** SWIAT=1, POSTACIE=2, HITBOX_WROGA=4, HITBOX_GRACZA=8, INTERAKCJE=16, Przesuwny=32. Mury generatora w ciele „Sciany”, okna w osobnym „Okna” (naklejki szukają tylko „Sciany”).
- **Warstwy renderu kondygnacji:** parter 2, piętro 4 (światła mają cull mask, żeby nie przeciekały przez strop).

Konwencje kodu

- GDScript 4, wcięcia tabulatorami, identyfikatory i komentarze **po polsku** (bez polskich znaków w identyfikatorach).
- Komentarz `##` nad funkcją opisuje, **co i dlaczego** z perspektywy gry.
- **Typy jawne**, gdy wynik jest wariantem (np. `var x: float = d["k"]`) – inferencja (`:=`) z wariantu daje błąd parsowania.
- `Array[RID]` jako `exclude` zapytań fizyki: zbudować typowaną tablicę albo przypisać do `q.exclude` przez zmienną typu `Array[RID]`.
- Nowy skrypt z `class_name` lub nowy zasób → `godot --headless --path . --import`.
- Nie dodawaj zależności zewnętrznych – wszystko generowane kodem (modele z brył, tekstury i dźwięki z Pythona).
- Dane balansu w JSON / stałych (`const`) na górze plików, nie rozsiane po logice.

Testy

Uruchamianie

- Headless: `godot --headless --path . <scena> -- <argumenty>`.
- Z obrazem: `xvfb-run -a -s "-screen 0 1920x1080x24" godot --path . --rendering-driver opengl3 <scena> -- <argumenty>`.
- Przed testami z zapisem: `rm -rf ~/.local/share/godot/app_userdata/*/zapis_test.json*`.
- **Nowy tryb testowy rajdu dopisz do listy w `Gra._ready()`** (zapis testowy), inaczej nadpisze zapis gracza.
- **Nigdy nie puszczaj kilku testów godota naraz** – pod obciążeniem testy fizyki i AI dają fałszywe błędy.

Tryby (`scenes/rajd.tscn -- …`)

Tryb	Co sprawdza
`--test --ziarno 31`	generowanie i start rajdu
`--walka`, `--frakcje`	walka gracza, wojna frakcji
`--test_boss --boss`, `--test_reputacji`	Sęp, reputacja
`--test_sprawdz DIR`, `--test_mapa DIR`, `--test_pulapka DIR`	sprawdzanie magazynka, mapa i spray, pułapka z granatu
`--test_v14 DIR`, `--test_v15 DIR`, `--test_v16 DIR`	potykacz, ciała, tempo, drzwi ręką, klin, przedmioty przesuwne
`--test_nakladanie DIR`, `--test_zapiecia DIR`	siatki ekwipunku, gesty zapięć
`--ewakuacja --zrzuty_raport DIR`	ewakuacja i raport
`--soak 480 --ziarno 31`	8 min AI bez gracza (zacięcia, drzwi, przedmioty)
`--ekonomia --ziarno 5`	wartość łupu na mapę
`--vm DIR --animacje --<tryb>`	widok z pierwszej osoby: mechaniki, przeladowania, gesty, drzwi, granat, poddanie, potrzeby, plecaki, porzucenie, lup, meble, ruch_broni, zwloki, ziemia, schody, czynnosci (`--tylko id`), latarka_przeladowanie

Baza: `scenes/baza.tscn -- --test_kryjowki` (też klikanie jak gracz: [B], lista, stawianie), `--zrzuty DIR`. Menu: `scenes/menu.tscn -- --zrzuty DIR` (panel ustawień, pełny ekran). Generator: `tests/test_sciany.tscn -- --kadry DIR --ziarna N [--przeglad]`.

Regresja

`bash narzedzia/regresja.sh /tmp/reg > /tmp/reg.log` – ok. 1,5 h, w tle. Wynik: `grep -E "TEST|BŁĄD|SCRIPT ERROR|SOAK" /tmp/reg.log`. Puszczać przed każdym wydaniem większej zmiany. Autor może poprosić o przerwanie – wtedy ubić procesy godota i Xvfb.

Weryfikacja wizualna

Po zmianach w modelach, animacjach i generatorze zrób zrzuty (`_zapisz`, `_zapisz_kadr` w `scena_rajd.gd`), złóż je w arkusz (Python + PIL) i **obejrzyj**. Liczby w teście nie wystarczą (przykład: butelka przy piciu, przeładowanie z latarką – test przechodził, a obraz był zły).

Wydanie

1. Uzupełnij `docs/STAN_DEMO.md` (nagłówek wersji + wiersz w tabeli) i ewentualnie `docs/MECHANIKI.md`.
2. Commit na `main` ze stopką wymaganą przez sesję, push.
3. `bash narzedzia/wydanie.sh 0.16.7` – eksport Windows, zip `StrefaK12/StrefaK12.exe`, push na gałąź `wydania`, link: `https://github.com/budownik1203/k-12/raw/wydania/StrefaK12_v0.16.7_Windows.zip`.
4. **Sprawdź rozmiar i datę exe przed spakowaniem** (raz wydano paczkę ze starym plikiem).
5. Raport dla autora po polsku: co zmienione, link, co niepewne, propozycje.

Wersjonowanie: `0.MAJOR.POPRAWKA` – nowa porcja mechanik podbija środkową liczbę, poprawki ostatnią.

Znane pułapki techniczne

Problem	Rozwiązanie
Renderer Compatibility świeci obiekt najwyżej kilkoma–kilkunastoma światłami (`max_lights_per_object=16`, `max_renderable_lights=64`)	geometria per pomieszczenie; `_widocznosc_swiatel` w `scena_rajd.gd` (kondygnacja gracza, 40 najbliższych)
Świat bazy jest w SubViewport bez wejścia myszy	UI i wejście kryjówki w głównym oknie (`get_tree().root`, klasa `Przekaznik`)
Kontrolka na środku ekranu z domyślnym `MOUSE_FILTER_STOP` łapie kliknięcia w świecie	wszystkie nakładki HUD z `MOUSE_FILTER_IGNORE`
UI zamykane przy wciśniętym LPM (wytrych)	odblokować sterowanie dopiero po puszczeniu przycisku
Quady w generatorze mają lico zależne od kolejności wierzchołków	orientować jak `_scianka` (normalna do wnętrza); test `test_sciany`
Statyczne lambdy w autoloadach	sprzątać w `_exit_tree` (inaczej uszkodzenie sterty przy wyjściu)
`godot --export-release … | head`	`head` zabija godota – nie przepuszczać wyjścia eksportu przez `head`
Test losowy (AI, odłamki)	zatrzymać AI w kluczowym momencie, sprawdzać mechanizm, a nie przypadkowy wynik

Jak dodać…

Przedmiot: wpis w `assets/dane/przedmioty.json` (id, nazwa, typ, w, h, waga, cena, opis + blok typu), model w `scripts/core/modele_przedmiotow.gd` (`czesci`), ikona: `python3`

`narzedzia/generuj_ikony.py` (render) – sprawdzić `tests/render_ikon.tscn`; ewentualnie tabela łupu / oferta handlarza; regeneracja `06_Przedmioty` (`narzedzia/dokumentacja_przedmioty.py`).

Mebel kryjówki: `KATALOG` w `scripts/core/meble_kryjowki.gd` + model w `meble.json` / `assets/models/meble`.

Mechanikę: sprawdź styk z istniejącymi systemami (tabele w `docs/MECHANIKI.md`), dodaj test (tryb w `scena_rajd.gd` + lista zapisu testowego w `Gra`), opisz w STAN_DEMO i MECHANIKI.

Dokumentację PDF: źródła Markdown w `docs/dokumentacja/*.md` (blok metadanych `nr/tytul/podtytul/stempel` nad `---`), budowa: `python3 narzedzia/dokumentacja_pdf.py [katalog] [filtr]` (Chromium przez Playwright).

Otwarte sprawy (stan na v0.16.6)

- v0.16.6 wypchnięta na `main`, **niewydana** – regresja przerwana w 2/3; jedyny błąd: `--test_v14` „wróg nie zdjął pułapki” (szklanka na klamce) – do sprawdzenia, czy losowość, czy regresja.
- Zgłoszenia „blok w powietrzu” i „plakat wisi” – poprawki zabezpieczające, przyczyna niepotwierdzona na mapie autora (nieznane ziarno). Warto dodać ziarno rajdu do raportu albo HUD-u debug, żeby autor mógł je podać.